Assignment 2: Verification and Submission

GStar Bootcamp

Abstract

Independent references, numerical error, state passing, checker commands, and submission requirements.

Companions: Overview, Final KDA problem.

10 Verification, grading, and submission

Independent reference and intermediate checkpoints

The public checker uses an independent FP64 PyTorch recurrence for small cases. It checks outputs and final state even when an upstream FLA package is absent. State and output kernels are tested with independently computed intermediate inputs. This isolates later kernels from earlier incomplete subparts. WY checks still depend on your shared triangular inverse. Official point scoring and hidden tests are instructor release work.

The following cases are covered by the public checker. Future hidden tests may vary the same conditions.

Table 1. Independent reference and intermediate checkpoints.

Case	Purpose
$T = 1, 63, 64, 65, 127, 128, 129, 200$	Diagonal, chunk-boundary, and padding behavior
$D_k/D_v = 64/64, 128/64, 64/128, 128/128$	Correct axes and full tile coverage
Multiple batches and heads	Flattened batch/head indexing
Nonzero initial state	Inter-chunk contribution and state decay
Sequence split at 1, 63, or 65	State contract across calls
Zero/scalar/unequal channel gates	Model reductions and gate axis
$\beta = 0, \beta = 1$, repeated keys	Write strength and prediction timing
Strong negative prefixes	Overflow avoidance and finite results
Custom scale	Exactly one output scale; unscaled state update

Error metrics

For student tensor $\widehat{\mathbf{Y}}$ and reference $\mathbf{Y}$, report

$$E_{\max} = \max_i |\widehat{Y}_i - Y_i|, \tag{10.1}$$

$$E_{\mathrm{relRMS}} = \frac{\sqrt{\frac{1}{n}\sum_i (\widehat{Y}_i - Y_i)^2}}{\max\left(\sqrt{\frac{1}{n}\sum_i Y_i^2}, \epsilon\right)}. \tag{10.2}$$

Check finite values before calculating metrics. Outputs and final states are evaluated separately. The public checker contains provisional dtype-specific tolerances. Official thresholds require calibration on the course GPUs; the reported 1,000-point allocation is the curriculum rubric, and the current checker reports pass/fail rather than points. BF16 chunk products and sequential FP32 reductions can differ by normal rounding; exact bitwise equality is not required.

Partial credit

The per-problem steps define the point split. Numerical tests validate implementation subparts; requested explanations validate shape reasoning, state timing, and numerical stability. Correctness of the full composition is checked separately. Any required test failure produces a nonzero grader exit status.

Verification: passing state between segments

Let a sequence split after token m. The recurrence requires

$$(\mathbf{O}_{1:m}, \mathbf{S}_m) = f(\mathbf{X}_{1:m}, \mathbf{S}_0), \qquad (\mathbf{O}_{m+1:T}, \mathbf{S}_T) = f(\mathbf{X}_{m+1:T}, \mathbf{S}_m).$$

Concatenated outputs and the final state must match a single full call within the numerical tolerance, including when m is not a chunk boundary.

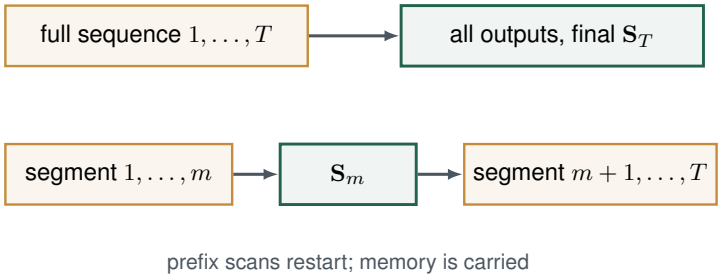


Figure 1. Streaming correctness checks the state interface. Restart local gate prefixes for the new call while carrying the completed matrix state.

```
# Supply matching gate/beta slices in each call as appropriate.
options = dict(scale=scale, output_final_state=True)
out_full, state_full = attention(q, k, v, ...,
                                 initial_state=s0, **options)
out_a, state_a = attention(q[:, :, :m], k[:, :, :m],
                           v[:, :, :m], ...,
                           initial_state=s0, **options)
out_b, state_b = attention(q[:, :, m:], k[:, :, m:],
                           v[:, :, m:], ...,
                           initial_state=state_a, **options)
# Compare cat(out_a, out_b) with out_full; state_b with state_full.
```

Executable commands and submission

Run these commands from the `GStar-Assignment-2` directory. Untouched templates intentionally fail correctness checks.

```
python autograder.py --p1 --device cpu  # PyTorch foundation
python autograder.py --p2               # CUDA; use --p3 through --p9
python autograder.py --all        # all required correctness tests
python autograder.py --p9 --benchmark  # timing after correctness passes
python autograder.py --check-templates  # syntax/API/dependencies only
```

Submission checklist

Submit the nine completed problem files and your requested solution comments. Reuse earlier student primitives. Forward kernels must use Triton; they must not call instructor attention operators or FLA attention kernels. Problem 1 and the supplied direct baseline are the specified PyTorch components. Backward kernels are optional future extensions.

- Preserve the public function signatures, return flags, and named student regions. Submit your implementations of `problem_1.py` through `problem_9.py`, together with the explanations requested in each problem.
- Make the final KDA path reuse P4's scan, P6's inverse, and P8's score builder. P6 reuses P3's output kernel. Explain these dependencies so a reviewer can identify the implemented path.
- Include correctness evidence for outputs and final states, not only a timing result. Record the input dtype and tested shapes, including unequal key/value dimensions, nonzero initial memory, and a split away from a chunk boundary.
- Record maximum absolute and relative RMS errors for both outputs and final state. Note which shared subpart is responsible when an isolated checkpoint fails. The public checker reports failures, not official point scores.

Timing after correctness

The `-benchmark` command runs timing only after the selected forward path passes its numerical checks. It excludes compilation and warmup and uses CUDA events. Current measurements include validation and allocation costs in the wrapper. Record GPU, dtype and toolchain; compare operators with matching gates, scale and initial states. Official speed thresholds and numerical tolerances require course GPU calibration.